

Express

Overview

- Minimal node js framework
- Features
 - complex routing
 - req/resp handling
 - middleware
 - server side rendering
- Rapid app development
 - MVC Architecture

basic app

```

1  const express = require('express');
2
3  const app = express();
4
5  app.get('/', (req, res) => {
6    console.log('Request received');
7    res.status(200).send('Hello from the server');
8  });
9
10 const port = 3000;
11 app.listen(port, () => {
12   console.log('App running on port ${port}');
13 });

```

Rest Client

Visual Studio | Marketplace

Visual Studio Code > Programming Languages > REST Client



REST Client

Huachao Mao |  5,893,395 installs | ★★★★★ (371) | Free

REST Client for Visual Studio Code

[Install](#)

[Trouble Installing?](#) 

Routing

<https://expressjs.com/en/guide/routing.html>

https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/routes

Cos'è il routing

- Il **routing** è il modo in cui un'applicazione risponde alle richieste HTTP su determinati percorsi (URL) con metodi specifici (GET, POST, ecc.).

Routing con stringhe

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Homepage');
});

app.get('/about', (req, res) => {
  res.send('Pagina About');
});

app.get('/contact', (req, res) => {
  res.send('Contattaci');
});

app.listen(3000, () => {
  console.log('Server in ascolto su http://localhost:3000');
});
```

Routing con stringhe

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Homepage');
});

app.get('/about', (req, res) => {
  res.send('Pagina About');
});

app.get('/contact', (req, res) => {
  res.send('Contattaci');
});

app.listen(3000, () => {
  console.log('Server in ascolto su http://localhost:3000');
});
```

- `app.get(path, callback)` → risponde a una richiesta GET sull'URL path
- `req` e `res` sono gli oggetti della richiesta e della risposta
- Le stringhe come `'/'`, `'/about'` sono chiamate **percorsi statici**
- Le rotte sono **case-sensitive** e **order-sensitive**
 - `/about` ≠ `/About`
- Le rotte sono lette nell'ordine in cui sono scritte

Routing sui metodi

```
// GET method route
app.get('/', (req, res) => {
  res.send('GET request to the homepage')
})

// POST method route
app.post('/', (req, res) => {
  res.send('POST request to the homepage')
})
```

Routing sui metodi

```
// GET method route
app.get('/', (req, res) => {
  res.send('GET request to the homepage')
})

// POST method route
app.post('/', (req, res) => {
  res.send('POST request to the homepage')
})
```

checkout(), copy(), **delete()**, **get()**, head(), lock(), merge(), mkactivity(), mkcol(), move(),
m-search(), notify(), options(), patch(), **post()**, purge(), **put()**, report(), search(),
subscribe(), trace(), unlock(), unsubscribe().

Routing sui metodi

```
// GET method route
app.get('/', (req, res) => {
  res.send('GET request to the homepage')
})
```

```
// POST method route
app.post('/', (req, res) => {
  res.send('POST request to the homepage')
})
```

checkout(), copy(), **delete()**, **get()**, head(), lock(), merge(), mkactivity(), mkcol(), move(),
m-search(), notify(), options(), patch(), **post()**, purge(), **put()**, report(), search(),
subscribe(), trace(), unlock(), unsubscribe().

```
app.all('/secret', (req, res, next) => {
  console.log('Accessing the secret section ...')
  next() // pass control to the next handler
})
```

Route parameters

- I **route parameters** (o **parametri dinamici**) sono segmenti della URL che fungono da **segnaposto** per valori variabili. Si usano per creare rotte **dinamiche** in Express.

Route parameters

- I **route parameters** (o **parametri dinamici**) sono segmenti della URL che fungono da **segnaposto** per valori variabili. Si usano per creare rotte **dinamiche** in Express.

```
const express = require('express');
const app = express();

app.get('/users/:userId/books/:bookId', (req, res) => {
  res.send(req.params);
});

app.listen(3000, () => {
  console.log('Server avviato su http://localhost:3000');
});
```

Route parameters

- I **route parameters** (o **parametri dinamici**) sono segmenti della URL che fungono da **segnaposto** per valori variabili. Si usano per creare rotte **dinamiche** in Express.

```
const express = require('express');
const app = express();

app.get('/:userId/books/:bookId', (req, res) => {
  res.send(req.params);
});

app.listen(3000, () => {
  console.log('Server avviato su http://localhost:3000');
});
```

Route parameters

- I **route parameters** (o **parametri dinamici**) sono segmenti della URL che fungono da **segnaposto** per valori variabili. Si usano per creare rotte **dinamiche** in Express.

```
const express = require('express');
const app = express();

app.get('/:userId/books/:bookId', (req, res) => {
  res.send(req.params);
});

app.listen(3000, () => {
  console.log('Server avviato su http://localhost:3000');
});
```

<http://localhost:3000/users/34/books/8989>

Route parameters

- I **route parameters** (o **parametri dinamici**) sono segmenti della URL che fungono da **segnaposto** per valori variabili. Si usano per creare rotte **dinamiche** in Express.

```
const express = require('express');
const app = express();

app.get('/:userId/books/:bookId', (req, res) => {
  res.send(req.params);
});

app.listen(3000, () => {
  console.log('Server avviato su http://localhost:3000');
});
```

http://localhost:3000/users/34/books/8989



```
{
  "userId": "34",
  "bookId": "8989"
}
```

Order of routes

```
app.get("/", (req, res) => {
  res.send("Home page");
});

app.get("/page", (req, res) => {
  res.send("A static page");
});

app.get("/:post", (req, res) => {
  res.send("Single post");
});

app.get("*", (req, res) => {
  res.send("Any");
});
```



```
app.get("/", (req, res) => {
  res.send("Home page");
});

app.get("/:post", (req, res) => {
  res.send("Single post");
});

app.get("/page", (req, res) => {
  res.send("A static page");
});

app.get("*", (req, res) => {
  res.send("Any");
});
```

- parametric path inserted just before a literal one takes the precedence over the literal one

route()

- È possibile creare handler di route concatenabili per un percorso di route, utilizzando **app.route()**.

```
app.route('/book')
  .get(function(req, res) {
    res.send('Get a random book');
  })
  .post(function(req, res) {
    res.send('Add a book');
  })
  .put(function(req, res) {
    res.send('Update the book');
  });
```

Routers

```
var express = require('express');
var router = express.Router();

// middleware that is specific to this router
router.use(function timeLog(req, res, next) {
  console.log('Time: ', Date.now());
  next();
});

// define the home page route
router.get('/', function(req, res) {
  res.send('Birds home page');
});

// define the about route
router.get('/about', function(req, res) {
  res.send('About birds');
});

module.exports = router;
```

La classe `express.Router` crea handler di route modulari e montabili.

Un'istanza Router è un middleware e un sistema di routing completo; per questa ragione, spesso si definisce "mini-app".

`/birds/`

`/birds/about`

```
var birds = require('./birds');
...
app.use('/birds', birds);
```

RESPONSE

Oggetto res

- `res` (response) rappresenta la risposta HTTP che inviamo al client

Metodo Express	Tipo di risposta	Descrizione
<code>res.send()</code>	testo / oggetto / buffer	Risposta generica, si adatta al tipo
<code>res.json()</code>	JSON	Risposta in formato JSON
<code>res.sendFile()</code>	file	Invia un file al client
<code>res.redirect()</code>	redirect HTTP	Reindirizza verso un altro URL
<code>res.status()</code>	imposta codice HTTP	Imposta lo stato della risposta

Esempi pratici

- `res.send()` con testo:

```
res.send('Ciao dal server!');
```

- `res.send()` con HTML:

```
res.send('<h1>Homepage</h1>');
```

- `res.json()`:

```
res.json({ user: 'Mario', id: 42 });
```

- `res.status()` + `res.send()`:

```
res.status(404).send('Pagina non trovata');
```

Impostare gli header HTTP

- Specificano **il tipo di contenuto**
- Permettono di inviare **informazioni extra** (token, CORS, cookie, ecc.)

```
res.set('Content-Type', 'text/plain');
res.send('Testo semplice');
```

```
res.set({
  'Content-Type': 'application/json',
  'X-Powered-By': 'Express'
});
res.send({ msg: 'Ok' });
```

send file

```
const app = express();

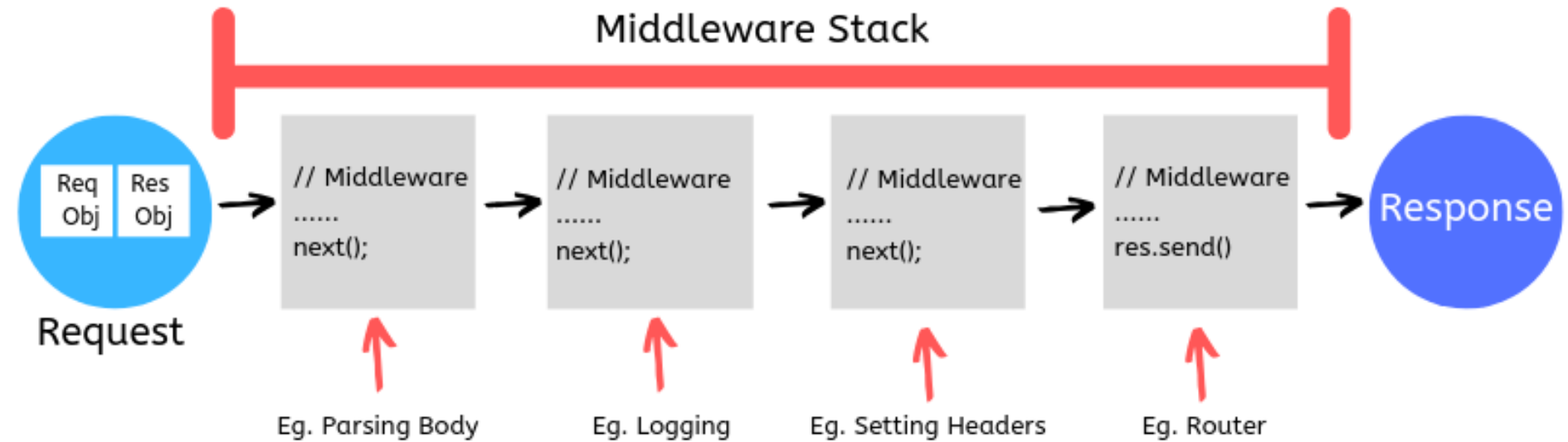
app.get('/', (req, res) => {
  res.status(200).sendFile(path.join(__dirname, '/index.html'));
});

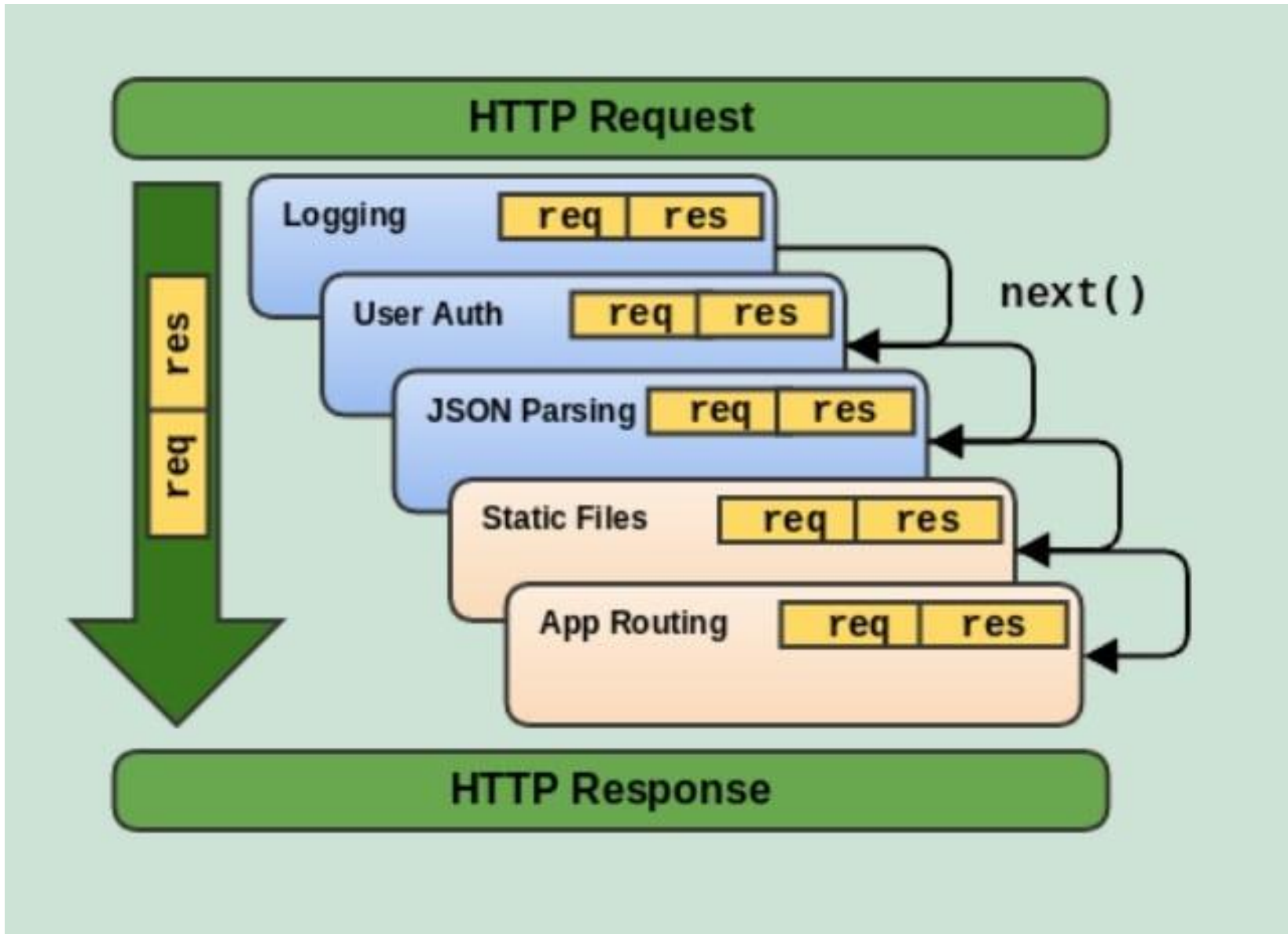
const port = 3000;

app.listen(port, () => {
  console.log(`App running on port ${port}`);
});
```

MIDDLEWARE

Middleware





Static Files

```
app.use(express.static('public'));
```

Per gestire i file statici, quali immagini, file CSS e file JavaScript, utilizzare la funzione middleware integrata `express.static` in Express.

Fornire il nome della directory che contiene gli asset statici alla funzione middleware `express.static` per iniziare a gestire i file direttamente. Ad esempio, utilizzare il seguente codice per gestire le immagini, i file CSS e i file JavaScript nella directory denominata `public`:

```
app.use(express.static('public'));
app.use(express.static('images'));
```

```
app.use('/static', express.static('public'));
```

Custom Middleware

```
app.use(function (req, res, next) {
  console.log('Hello from the middleware !');
  next();
});

app.use('/api', function (req, res, next) {
  console.log('This middleware handles the data route');
  next();
});
```

Third Party middleware

```
const morgan = require('morgan');  
app.use(morgan('dev'));
```

npm install morgan

<https://github.com/expressjs/morgan>

param()

```
const express = require("express");
const router = express.Router();

router.param("userId", (req, res, next, id) => {
  console.log("This function will be called first");
  next();
});

router.get("/user/:userId", (req, res) => {
  console.log("Then this function will be called");
  res.end();
});

// Export router
module.exports = router;
```

multiple handlers

```
var cb0 = function (req, res, next) {
  console.log('CB0');
  next();
}

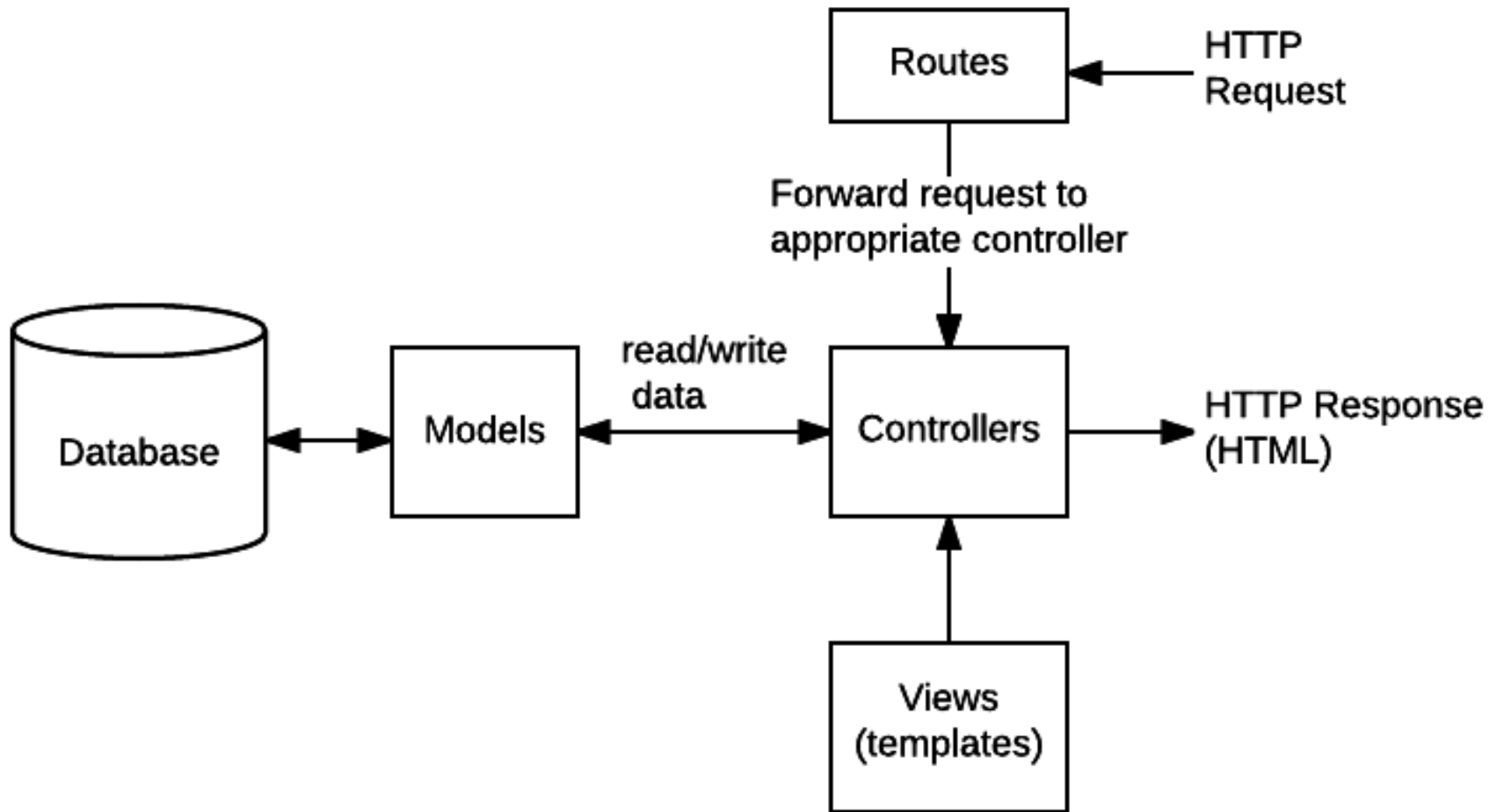
var cb1 = function (req, res, next) {
  console.log('CB1');
  next();
}

var cb2 = function (req, res) {
  res.send('Hello from C!');
}

app.get('/example/c', [cb0, cb1, cb2]);
```

Un array di funzioni callback possono gestire una route.

MVC Architecture



Environment

- Environment variables allowing apps to behave differently based on the environment
- Externalize all environment specific parameters
- Examples
 - Which HTTP port to listen on
 - What path and folder your files are located in, that you want to serve
 - Pointing to a development, staging, test, or production database

Examples

```
const port = process.env.PORT;

app.listen(port, () => {
  console.log(`App running on port ${port}`);
});
```

- PORT=8626 node server.js
- PORT=8626 NODE_ENV=development node server.js

```
console.log(app.get('env'));

if (app.get('env') == 'development') {
  app.use(morgan('dev'));
}
```

.env file

.env

```
PORT=8765  
NODE_ENV=development  
  
USERNAME=loreti  
PASSWORD=12345
```

server.js

```
const dotenv = require('dotenv');  
dotenv.config();
```

```
const dotenv = require('dotenv');  
dotenv.config({ path: '/custom/path/to/.env' });
```

config module

config.js

```
const dotenv = require('dotenv');  
dotenv.config();  
  
module.exports = {  
  username: process.env.USER_NAME,  
  password: process.env.PASSWORD,  
  port: process.env.PORT,  
};
```

server.js

```
const { port, username, password } = require('./config');
```